

编译原理期末

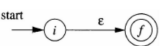
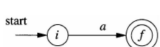
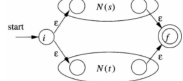
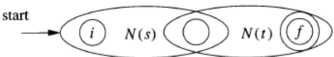
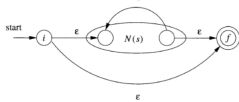
题目1 正则表达式与自动机

NFA 简洁易于理解, 便于描述语言 $L(A)$

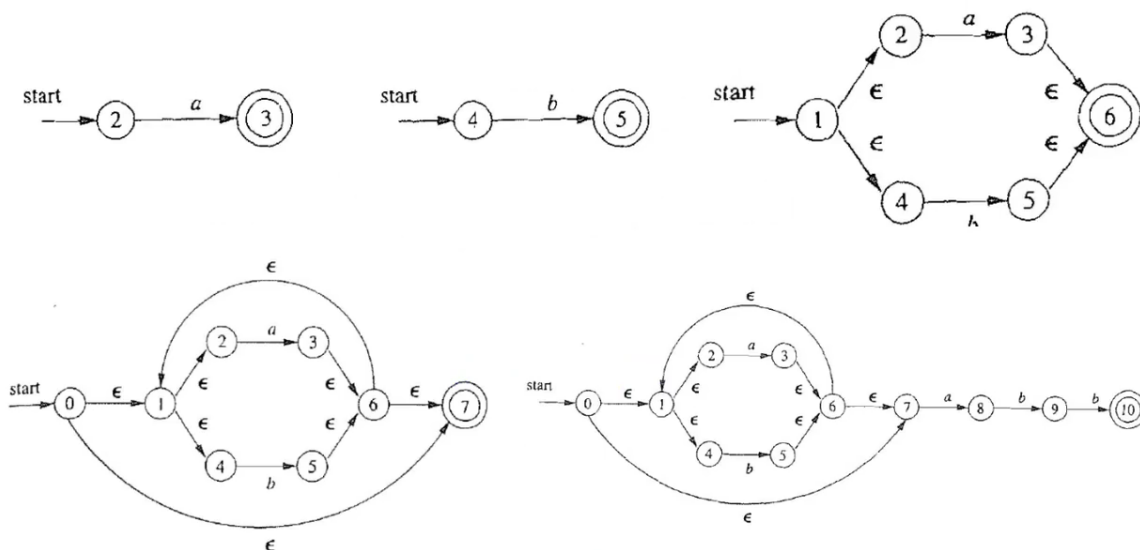
DFA 易于判断 $x \in L(A)$, 适合产生词法分析器

Thompson构造法

Thompson构造法的基本思想: 按结构归纳

Definition (正则表达式) 给定字母表 Σ, Σ 上的正则表达式由以下规则定义: (1) ϵ 是正则表达式; (2) $\forall a \in \Sigma$, a 是正则表达式; (3) 如果 s 是正则表达式, 则 (s) 是正则表达式; (4) 如果 s 与 t 是正则表达式, 则 $s t$, st, s^* 也是正则表达式。	表达式 ϵ,  表达式 a, 	正则表达式 s 和 t 的 NFA 分别是 $N(s)$ 和 $N(t)$ $r=s t$, r 的 NFA $N(r)$ 
归纳规则 正则表达式 $r=st$, $N(r)$  图 3-41 两个正则表达式的连接的 NFA	归纳规则 正则表达式 $r=s^*$, $N(r)$  图 3-42 一个正则表达式的闭包的 NFA	例题如下

$$r = (a|b)^*abb$$



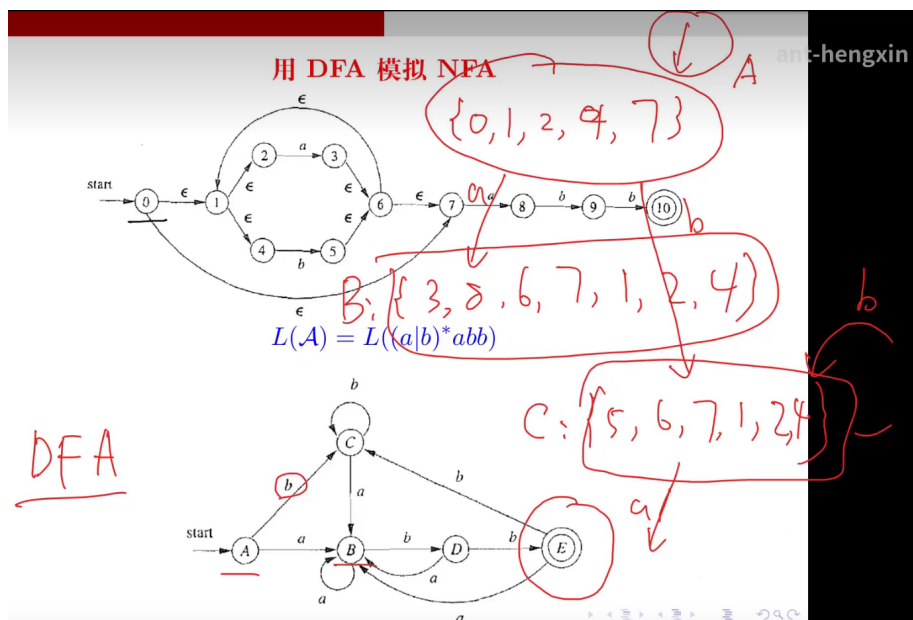
注意写出单个字母的 NFA

子集构造法

思想: 用 DFA 模拟 NFA

模拟过程:

1. 确定初始状态, 从 NFA 的初始状态及其 ϵ 闭包开始
2. 状态转移



3. 确定结束状态, 某一个状态的状态子集中含有 NFA 中的结束状态

NFA 状态	DFA 状态	a	b
$\{0, 1, 2, 4, 7\}$	A	B	C
$\{1, 2, 3, 4, 6, 7, 8\}$	B	B	D
$\{1, 2, 4, 5, 6, 7\}$	C	B	C
$\{1, 2, 4, 5, 6, 7, 9\}$	D	B	E
$\{1, 2, 4, 5, 6, 7, 10\}$	E	B	C

开放知识: 存在这样的 NFA , 其对应的 DFA 状态数呈指数级增长

形式化表达:

操作	描述
$\epsilon\text{-closure}(s)$	能够从 NFA 的状态 s 开始只通过 ϵ 转换到达的 NFA 状态集合
$\epsilon\text{-closure}(T)$	能够从 T 中某个 NFA 状态 s 开始只通过 ϵ 转换到达的 NFA 状态集合, 即 $\cup_{s \in T} \epsilon\text{-closure}(s)$.
$move(T, a)$	能够从 T 中某个状态 s 出发通过标号为 a 的转换到达的 NFA 状态的集合

图 3-31 NFA 状态集上的操作

DFA最小化

DFA 最小化算法基本思想: 等价的状态可以合并

如何定义等价状态 (反面, 如何判断不等价):

$$s \approx t \iff \exists a \in \Sigma. (s \xrightarrow{a} s') \wedge (t \xrightarrow{a} t') \wedge (s' \approx t')$$

从哪里开始呢? 接受状态与非接受状态必定不等价

模拟过程

1. 分裂状态

① $bQ \{A, B, C, D\} \xrightarrow{b} \{E\}$

$$s \approx t \iff \exists a \in \Sigma. (s \xrightarrow{a} s') \wedge (t \xrightarrow{a} t') \wedge (s' \approx t')$$

从哪里开始呢?

② $\{A, B, C\} \xrightarrow{b} \{D\} \xrightarrow{b} \{E\}$

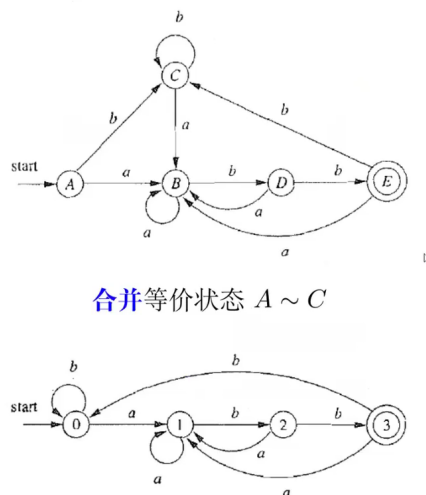
$\Pi = \{F, S \setminus F\}$

接受状态与非接受状态必定不等价

③ ④

魏恒峰 (hfw@nju.edu.cn) 词法分析 2023 年 03 月 10 日 (周五) 50 / 78

2. 合并等价状态



注意：这是DFA最小化算法，所以要先检查是否需要补“死状态”

例题

一、

题目 1 (正则表达式与自动机)

考虑正则表达式 $r = (1 \mid 01)^* 0^*$ (字母表 $\Sigma = \{0, 1\}$)。①

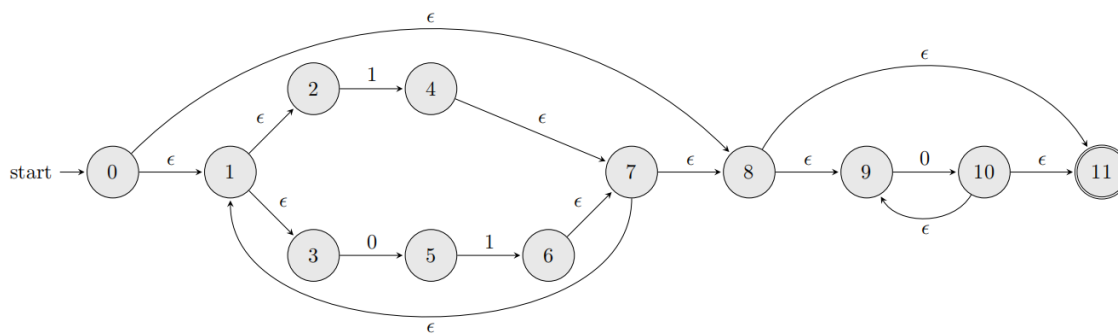
- (1) 使用 Thompson 构造法构造等价的 NFA;
- (2) 使用子集构造法构造等价的 DFA;
- (3) 将上一步构造的 DFA 最小化;

以上各小题，请给出关键的中间步骤。

(不必给出所有的细节，类似的步骤可以“跳步”)

解答：

1.

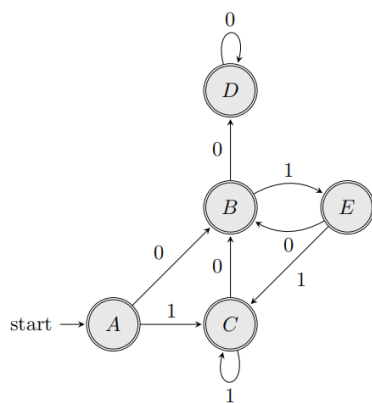


2.

NFA和DFA状态

NFA 状态	DFA 状态	0	1
0,1,2,3,8,9,11	A	B	C
5,9,10,11	B	C	C
1,2,3,4,7,8,9,11	C	B	C
9,10,11	D	D	$\emptyset$
1,2,3,6,7,8,9,11	E	B	C

DFA



3.

$$\Pi_0 = \{\{A, B, C, D, E\}\}$$

$$\Pi_1 = \{\{A, B, C, E\}, \{D\}\}$$

$$\Pi_2 = \{\{A, C, E\}, \{B\}, \{D\}\}$$

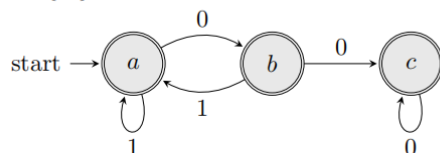
$$\Pi_3 = \Pi_2 = \Pi_{final}$$

或者直接看出来合并 A,C,E

$$a = \{A, C, E\}$$

$$b = \{B\}$$

$$c = \{D\}$$



二、2020年作业

题目 1 (正则表达式与自动机 [10 = 2 + 2 + 2 + 2 + 2 分])

考虑正则表达式 $r = a(b|c)^*$ 。①

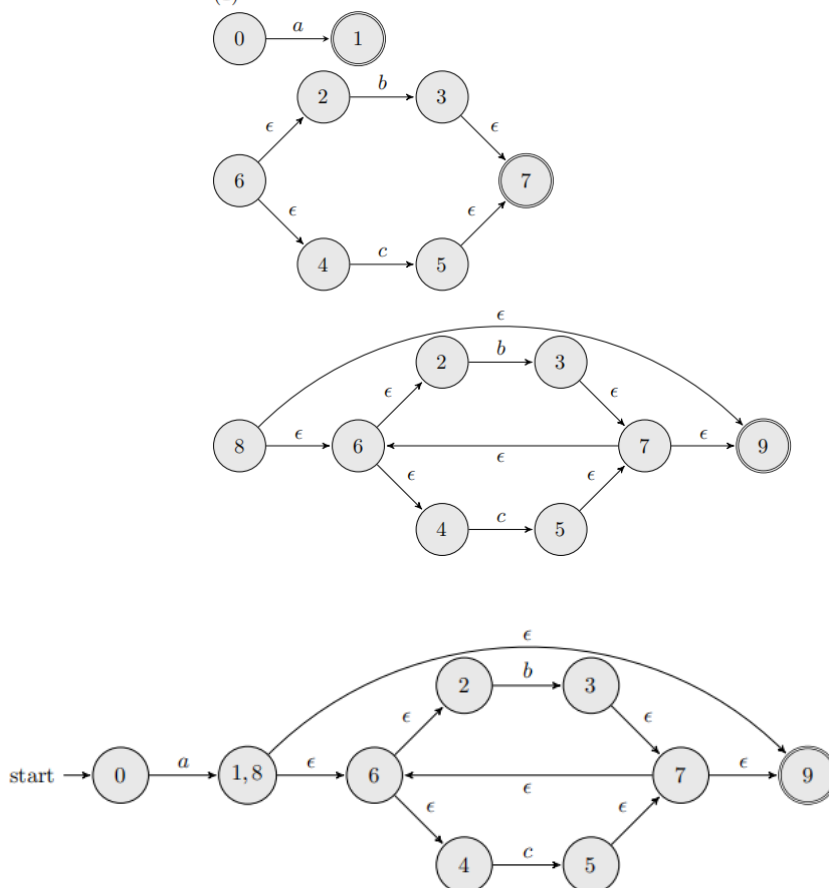
(1) 使用 Thompson 构造法构造等价的 NFA; ②

(2) 使用子集构造法构造等价的 DFA;

(3) 将上一步构造的 DFA 最小化;

解答:

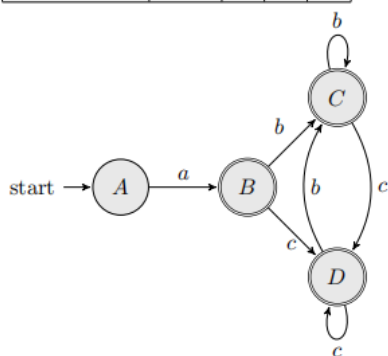
(1)



(2)

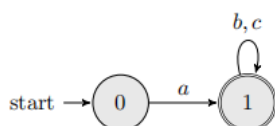
状态 1,8 用 1 表示

NFA	DFA	a	b	c
{0}	A	B	$\emptyset$	$\emptyset$
{1,2,4,6,9}	B	$\emptyset$	C	D
{2,3,4,6,7,9}	C	$\emptyset$	C	D
{2,4,5,6,7,9}	D	$\emptyset$	C	D



(3)

初始划分 $\{A, \{B, C, D\}\}$, 且 $\{B, C, D\}$ 对于输入 a,b,c 有相同转移, 不可分。则最终划分为 $\{A, \{B, C, D\}\}$ 。



题目2 LL语法分析

基本方法

每个中间节点表示对某个非终结符应用某个产生式进行推导

Q : 选择哪个非终结符, 以及选择哪个产生式

在推导的每一步, $LL(1)$ 总是选择最左边的非终结符进行展开

$LL(1)$: 从左向右读入词法单元

递归下降

递归下降的典型实现框架

```
void A() {  
    1) 选择一个 A 产生式,  $A \rightarrow X_1 X_2 \dots X_k$ ;  
    2) for (  $i = 1$  to  $k$  ) {  
    3)     if (  $X_i$  是一个非终结符号 )  
    4)         递归下降调用其它非终结符对应的递归函数  
    5)         调用过程  $X_i()$ ;  
    6)     else if (  $X_i$  等于当前的输入符号  $a$  )  
    7)         读入下一个输入符号;  
    8)     else /* 发生了一个错误 */;  
    9)         出现了不期望出现的词法单元  
    }  
}
```

为每个非终结符写一个递归函数

内部按需调用其它非终结符对应的递归函数, 下降一层

预测分析技术

使用预测分析表确定产生式

$$\begin{aligned} S &\rightarrow F \\ S &\rightarrow (S + F) \\ F &\rightarrow a \end{aligned}$$

	(	)	a	+	$\$$
S	2		1		
F			3		

指明了每个非终结符在面对不同的词法单元或文件结束符时, 该选择哪个产生式 (按编号进行索引) 或者报错 (空单元格)

$LL(1)$ 文法定义

Definition ($LL(1)$ 文法)

如果文法 G 的预测分析表是无冲突的, 则 G 是 $LL(1)$ 文法。

FIRST集合和FOLLOW集合

FIRST集合

FIRST(α) 是可从 α 推导得到的句型的**首终结符号**的集合

Definition (FIRST(α) 集合)

对于任意的 (产生式的右部) $\alpha \in (N \cup T)^*$:

$$\text{FIRST}(\alpha) = \{t \in T \cup \{\epsilon\} \mid \alpha \xRightarrow{*} t\beta \vee \alpha \xRightarrow{*} \epsilon\}.$$

FOLLOW集合

FOLLOW(A) 是可能在某些句型中**紧跟在 A 右边的终结符**的集合

Definition (FOLLOW(A) 集合)

对于任意的 (产生式的左部) 非终结符 $A \in N$:

$$\text{FOLLOW}(A) = \{t \in T \cup \{\$\} \mid \exists s. S \xRightarrow{*} s \triangleq \beta A t \gamma\}.$$

考虑产生式 $A \rightarrow \alpha$,

如果从 α 可能推导出空串 ($\alpha \xRightarrow{*} \epsilon$),

则只有当当前词法单元 $t \in \text{FOLLOW}(A)$, 才可以选择该产生式

如何计算FIRST集合

算法	示例
先计算每个符号 X 的 FIRST(X) 集合 <pre> 1: procedure FIRST(X) 2: if $X \in T$ then ▷ 规则 1: X 是终结符 3: FIRST(X) = X 4: for $X \rightarrow Y_1 Y_2 \dots Y_k$ do ▷ 规则 2: X 是非终结符 5: FIRST(X) ← FIRST(X) ∪ {FIRST(Y_1) \ {ϵ}} 6: for $i \leftarrow 2$ to k do 7: if $\epsilon \in L(Y_1 \dots Y_{i-1})$ then 8: FIRST(X) ← FIRST(X) ∪ {FIRST(Y_i) \ {ϵ}} 9: if $\epsilon \in L(Y_1 \dots Y_k)$ then ▷ 规则 3: X 可推导出空串 10: FIRST(X) ← FIRST(X) ∪ {ϵ}</pre> 不断应用上面的规则, 直到每个 FIRST(X) 都不再变化 再计算每个符号串 α 的 FIRST(α) 集合 $\alpha = X\beta$ $\text{FIRST}(\alpha) = \begin{cases} \text{FIRST}(X) & \epsilon \notin L(X) \\ (\text{FIRST}(X) \setminus \{\epsilon\}) \cup \text{FIRST}(\beta) & \epsilon \in L(X) \end{cases}$ 最后, 如果 $\epsilon \in L(\alpha)$, 则将 ϵ 加入 FIRST(α)。	Handwritten example of FIRST set calculation: - Grammar rules: (1) $X \rightarrow Y$ (2) $X \rightarrow a$ (3) $Y \rightarrow \epsilon$ (4) $Y \rightarrow c$ (5) $Z \rightarrow d$ (6) $Z \rightarrow XYZ$ - Handwritten calculations: $F(X) = \{a, c, \epsilon\}$ $F(Y) \subseteq F(X) \Rightarrow F(Y) = \{\epsilon\}$ $F(Z) = \{d, a, c\}$ $F(Z) \subseteq F(Z) \Rightarrow F(Z) = \{a, c, d, \epsilon\}$ - Question: $\epsilon \in F(Z)?$

如何计算FOLLOW集合

算法	示例
为每个非终结符 X 计算 $\text{FOLLOW}(X)$ 集合 <pre> 1: procedure FOLLOW(X) 2: for X 是开始符号 do ▷ 规则 1: X 是开始符号 3: $\text{FOLLOW}(X) \leftarrow \text{FOLLOW}(X) \cup \{\\$ \}$ 4: for $A \rightarrow \alpha X$ do ▷ 规则 2: X 是某产生式右部的最后一个符号 5: $\text{FOLLOW}(X) \leftarrow \text{FOLLOW}(X) \cup \text{FOLLOW}(A)$ 6: for $A \rightarrow \alpha X \beta$ do ▷ 规则 3: X 是某产生式右部中间的一个符号 7: $\text{FOLLOW}(X) \leftarrow \text{FOLLOW}(X) \cup (\text{FIRST}(\beta) \setminus \{\epsilon\})$ 8: if $\epsilon \in \text{FIRST}(\beta)$ then 9: $\text{FOLLOW}(X) \leftarrow \text{FOLLOW}(X) \cup \text{FOLLOW}(A)$ </pre> 不断应用上面的规则, 直到每个 $\text{FOLLOW}(X)$ 都不再变化 (不动点!!!)	

预测分析表

如何根据 **FIRST** 与 **FOLLOW** 集合计算给定文法 G 的预测分析表?

对应每条产生式 $A \rightarrow \alpha$ 与终结符 t , 如果

$$t \in \text{FIRST}(\alpha) \quad (1)$$

$$\alpha \xRightarrow{*} \epsilon \wedge t \in \text{FOLLOW}(A) \quad (2)$$

则在表格 $[A, t]$ 中填入 $A \rightarrow \alpha$ (编号)。

上面例子的答案:

	a	c	d	$\$$
X	1, 2	1	1	1
Y	3	3, 4	3	3
Z	6	6	5, 6	

改造成LL(1)文法

消除左递归

左递归会产生什么问题:

E 在**不消耗任何词法单元**的情况下, 直接递归调用 E , 造成**死循环**

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid \text{id} \mid \text{num}$$

如何消除直接左递归:

直接左递归 (Direct Left Recursion)

$$A \rightarrow A\alpha \mid \beta$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

其中, β_i 都不以 A 开头

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \epsilon$$

如何消除间接左递归:

方法示例	答案
$S: A_1$ $A: A_2$ $S \rightarrow Ac \mid c$ $A \rightarrow Bb \mid b$ $B \rightarrow Sa \mid a$ ③ ① $B \rightarrow Sa$ $\rightarrow (Ac \mid c)a$ $\rightarrow \textcircled{A}ca \mid ca$ $\rightarrow (Bb \mid b)ca \mid ca$ $\rightarrow Bbca \mid bca \mid ca$ 用消除直接左递归的方法消除	$S \rightarrow Ac \mid c$ $A \rightarrow Bb \mid b$ $B \rightarrow Sa \mid a$ $B \rightarrow Sa \mid a$ $\rightarrow (Ac \mid c)a \mid a$ $\rightarrow \textcolor{red}{A}ca \mid ca \mid a$ $\rightarrow (Bb \mid b)ca \mid ca \mid a$ $\rightarrow \textcolor{red}{B}bca \mid bca \mid ca \mid a$ $S \rightarrow Ac \mid c$ $A \xrightarrow{b} Bb \mid b$ $B \rightarrow (bca \mid ca \mid a)B'$ $B' \rightarrow bcaB' \mid \epsilon$ $A_i \rightarrow A_j \alpha \Rightarrow i < j$

算法要求:

文法中不存在环 (形如 $A \xRightarrow{*} A$ 的推导)

文法中不存在 ϵ 产生式 (形如 $A \rightarrow \epsilon$ 的产生式)

提取左公因子

提取左公因子 (Left-Factoring)

```

7  /*
8  decl : 'int' ID ';'
9  decl : 'int' ID '=' ID ';'
10
11  */
12
13  decl : 'int' ID optional_init ';' ;
14  optional_init
15      : '=' ID # Init
16      |      # NoInit
17      ;
18
19  /*
20  decl : 'int' ID ('=' ID)? ';'
21  */

```

ANTLR 4 可以处理有左公因子的文法

提取左公因子无助于消除文法二义性

例题

题目 1

给定下述文法 G ,

$$S \rightarrow BAS \quad (1)$$

$$S \rightarrow cA \quad (2)$$

$$A \rightarrow aA \quad (3)$$

$$A \rightarrow e \quad (4)$$

$$B \rightarrow bB \quad (5)$$

$$B \rightarrow \epsilon \quad (6)$$

- (1) 请为该文法计算必要的 FIRST 集合;
- (2) 请为该文法计算必要的 FOLLOW 集合;
- (3) 请为该文法设计预测分析表;
- (4) 该文法是 $LL(1)$ 文法吗? 请说明理由;

参考答案:

	FIRST	FOLLOW
A	{ a, e }	{ a, b, c, \$, e }
B	{ λ, b }	{ a, e }
S	{ a, b, c, e }	{ \$ }

	a	b	c	e	\$
A	aA			e	
B	λ	bB		λ	
S	BAS	BAS	cA	BAS	

题目 1 ([10 = 2 + 2 + 2 + 1 + 3 分])

给定下述文法 G ,

$$S \rightarrow aAb \quad (1)$$

$$S \rightarrow bAa \quad (2)$$

$$A \rightarrow cS \quad (3)$$

$$A \rightarrow \epsilon \quad (4)$$

- (1) 为该文法计算必要的 FIRST 集合;
- (2) 为该文法计算必要的 FOLLOW 集合;
- (3) 为该文法设计预测分析表;
- (4) 该文法是 $LL(1)$ 文法吗? 请说明理由。
- (5) 如果该文法是 $LL(1)$ 文法, 请给出相应的 $LL(1)$ 语法分析器的伪代码 (可以使用递归下降实现框架, 也可以使用非递归的版本); 如果该文法不是 $LL(1)$ 文法, 请将其改造成 $LL(1)$ 文法 (不必再重复各小题)。

解答：

(1) $\text{FIRST}(S)=\{a, b\}$, $\text{FIRST}(A)=\{\epsilon, c\}$

(2) $\text{FOLLOW}(S)=\{\$, a, b\}$, $\text{FOLLOW}(A)=\{a, b\}$

(3)

	a	b	c	$\$$
S	$S \rightarrow aAb$	$S \rightarrow bAa$		
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	$A \rightarrow cS$	

(4) 是 LL(1) 文法，预测分析表没有冲突

(5)

Algorithm 1 Parser

```
1: procedure PARSE( $w, M$ )            $\triangleright w$  is lexical string,  $M$  is prediction table
2:    $input \leftarrow w\$, t \leftarrow input.first()$             $\triangleright \$$  is end character,  $t \in \{a, b, c, \$\}$ 
3:    $stack \leftarrow S\$, X \leftarrow stack.top()$             $\triangleright$  use stack to simulate,  $X \in \{a, b, c, A, S, \$\}$ 
4:   while  $X \neq \$$  do
5:     if  $X == t$  then            $\triangleright X$  is terminal and equals to  $t$ 
6:        $stack.pop()$ 
7:        $X = stack.top()$ 
8:        $t = input.next()$ 
9:     else if  $X \in \{a, b, c\}$  then            $\triangleright X$  is terminal and  $X \neq t$ 
10:       $error()$ 
11:     else if  $M[X, t]$  is empty then            $\triangleright$  error: no Production
12:       $error()$ 
13:     else if  $M[X, t] = X \rightarrow \gamma_1\gamma_2 \dots \gamma_n$  then            $\triangleright$  choose a Production
14:       $print(X \rightarrow \gamma_1\gamma_2 \dots \gamma_n)$ 
15:       $stack.pop()$ 
16:       $stack.push(\gamma_1\gamma_2 \dots \gamma_n)$ 
17:       $X = stack.top()$ 
18:     end if
19:   end while
20: end procedure
```

题目3 LR语法分析

LR语法分析简介

自底向上的、
不断归约的、
基于句柄识别自动机的、
适用于 **LR 文法** 的、
LR 语法分析器

自底向上 构建语法分析树

根节点 是文法的起始符号 S

每个 **中间非终结符节点** 表示 **使用它的某条产生式进行归约**

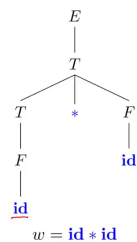
叶节点 是词法单元流 $w\$$

仅包含终结符号与特殊的 **文件结束符** $\$$

自顶向下的“推导”与 自底向上的“归约”

$$E \xRightarrow{rm} T \xRightarrow{rm} T * F \xRightarrow{rm} T * id \xRightarrow{rm} F * id \xRightarrow{rm} id * id$$

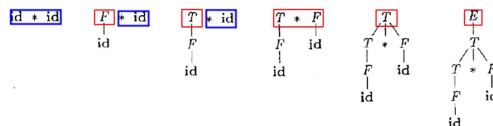
- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$



$$E \Leftarrow T \Leftarrow T * F \Leftarrow T * id \Leftarrow F * id \Leftarrow id * id$$

LR 语法分析器的状态

在任意时刻, 语法分析树的 **上边缘** 与 **剩余的输入** 构成当前句型



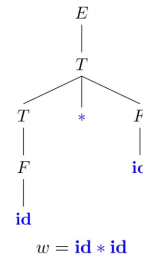
$$E \Leftarrow T \Leftarrow T * F \Leftarrow T * id \Leftarrow F * id \Leftarrow id * id$$

LR 语法分析器使用 **栈** 存储语法分析树的 **上边缘**

它包含了语法分析器目前所知的所有信息

演示“**栈**”上操作

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

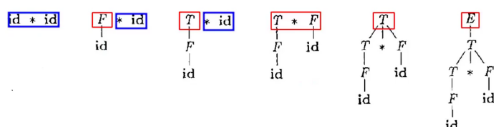


两大操作: **移入输入符号** 与 **按产生式归约**

直到栈中仅剩开始符号 E , 且输入已结束, 则成功停止

LR 语法分析器的状态

在任意时刻, 语法分析树的 **上边缘** 与 **剩余的输入** 构成当前句型



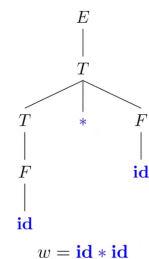
$$E \Leftarrow T \Leftarrow T * F \Leftarrow T * id \Leftarrow F * id \Leftarrow id * id$$

LR 语法分析器使用 **栈** 存储语法分析树的 **上边缘**

它包含了语法分析器目前所知的所有信息

演示“**栈**”上操作

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$



两大操作: **移入输入符号** 与 **按产生式归约**

直到栈中仅剩开始符号 E , 且输入已结束, 则成功停止

基于栈的 LR 语法分析器

Q_1 : 何时归约? (何时移入?)

Q_2 : 按哪条产生式进行归约?

LR (实际为 SLR) 分析表指导 LR 语法分析器

状态	ACTION					GOTO			
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2	r2	s7		r2		r2			
3	r4	r4		r4		r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7	r1		r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

在当前状态 (编号) 下, 面对当前文法符号时, 该采取什么动作

ACTION 表指明动作, GOTO 表仅用于归约时的状态转换

sn	移入输入符号, 并进入状态 n
rk	使用 k 号产生式进行归约
gn	转换到状态 n
acc	成功接受, 结束
空白	错误

GOTO 栏中符号的前缀都是 g , 例如在 4 号状态进行归约后压入 T , 此时会跳转到 2 号状态

LR 分析表与栈上操作

演示“栈”上操作: 移入与归约

状态	ACTION					GOTO			
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2	r2	s7		r2		r2			
3	r4	r4		r4		r4			
4	s5			s4			8	2	3
5	r6	r6		r6		r6			
6	s5			s4				9	3
7	s5			s4					10
8	s6			s11					
9	r1	s7		r1		r1			
10	r3	r3		r3		r3			
11	r5	r5		r5		r5			

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $E \rightarrow (E)$

(6) $F \rightarrow id$

$w = id * id \$$

栈中存储语法分析器的状态 (编号), “编码”了语法分析树的上边缘

注意:

- 规约中, 弹出符号时自动机的状态已经改变
- 规约不改变输入字符流的指针
- 移入后输入字符流的指针向后移一位

Definition (句柄 (Handle))

在输入串的 (唯一) 反向最右推导中, **如果** 下一步是逆用产生式 $A \rightarrow \alpha$ 将 α 归约为 A , 则称 α 是**当前句型的句柄**。

存在一种 LR 语法分析方法, 保证**句柄总是出现在栈顶**。

$LR(0)$ 句柄识别有穷状态机

状态刻画了“当前观察到的**针对所有产生式的右部的**前缀”

Definition ($LR(0)$ 项 (Item))

文法 G 的一个 $LR(0)$ **项**是 G 的某个产生式加上一个位于体部的**点**。

项指明了语法分析器已经观察到了某个产生式的某个前缀

$$A \rightarrow XYZ$$

$$[A \rightarrow \cdot XYZ]$$

$$[A \rightarrow X \cdot YZ]$$

$$[A \rightarrow XY \cdot Z]$$

$$[A \rightarrow XYZ \cdot]$$

(产生式 $A \rightarrow \epsilon$ 只有一个项 $[A \rightarrow \cdot]$)

项集就是若干**项**构成的集合

因此, 句柄识别自动机的一个**状态**可以表示为一个**项集**

Definition (增广文法 (Augmented Grammar))

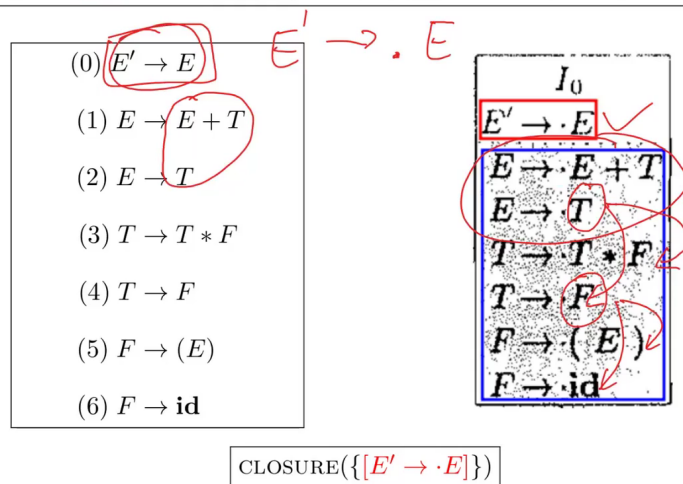
文法 G 的**增广文法** G' 是在 G 中加入产生式 $S' \rightarrow S$ 得到的文法。

目的: 告诉语法分析器何时停止分析并接受输入符号串

语法分析器**当前栈中仅有 S 且面对 $\$$** ,
要使用 $S' \rightarrow S$ 进行归约时, 输入符号串被接受

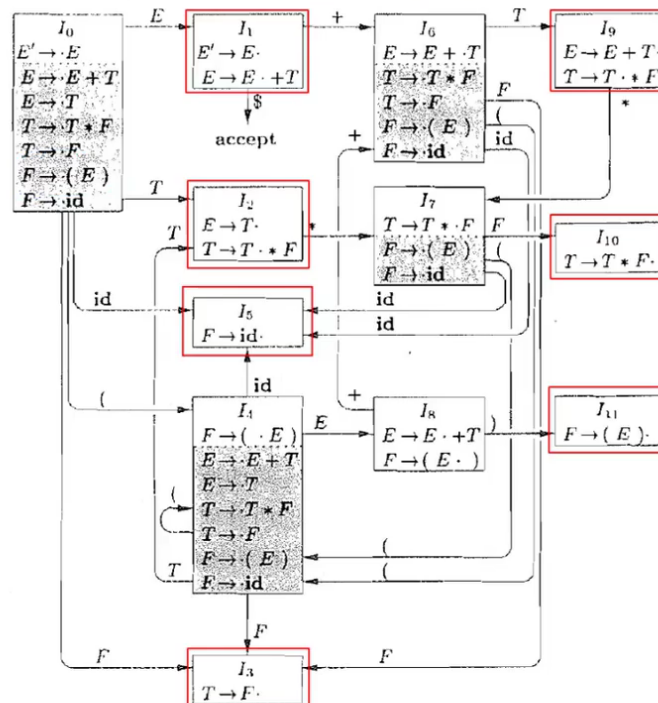
得到初始状态:

点指示了栈顶, 左边 (与路径) 是栈中内容, 右边是期望看到的文法符号串



如何生成其他状态:

演示 LR(0) 句柄识别自动机的构造过程



$$J = \text{GOTO}(I, X) = \text{CLOSURE}\left(\left\{[A \rightarrow \alpha X \cdot \beta] \mid [A \rightarrow \alpha \cdot X \beta] \in I\right\}\right)$$

接受状态:

接受状态: $F = \{I \in C \mid \exists [A \rightarrow \alpha \cdot] \in I\}$

(此时, 产生式 $A \rightarrow \alpha$ 的完整右部出现在栈顶)

LR(0)分析表

LR(0)分析表

LR(0) 分析表

GOTO 函数被拆分成 ACTION 表 (针对终结符) 与 GOTO 表 (针对非终结符)

如何规约

② $E \rightarrow T$.

(3) $[k : A \rightarrow \alpha \cdot] \in I_i \wedge A \neq S' \Rightarrow \forall t \in T \cup \{\$ \}. ACTION[i, t] = rk$

LR(0) 分析表构造规则总结

- $GOTO(I_i, a) = I_j \wedge a \in T \Rightarrow ACTION[i, a] \leftarrow sj$
- $GOTO(I_i, A) = I_j \wedge A \in N \Rightarrow GOTO[i, A] \leftarrow gj$
- $[k : A \rightarrow \alpha \cdot] \in I_i \wedge A \neq S' \Rightarrow \forall t \in T \cup \{\$ \}. ACTION[i, t] = rk$
- $[S' \rightarrow S \cdot] \in I_i \Rightarrow ACTION[i, \$] \leftarrow acc$

如果文法G的LR(0)分析表是**无冲突的**，则G是LR(0)文法

LR(0) 分析表每一行 (状态) **所选用的归约产生式是相同的**

	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			g1	g2	g3
1		s6				acc			
2	r2	r2	s7, r2	r2	r2	r2			
3	r4	r4	r4	r4	r4	r4			
4	s5			s4			g8	g2	g3
5	r6	r6	r6	r6	r6	r6			
6	s5			s4			g9	g3	
7	s5			s4			g10		
8		s6				s11			
9	r1	r1	s7, r1	r1	r1	r1			
10	r3	r3	r3	r3	r3	r3			
11	r5	r5	r5	r5	r5	r5			

归约时不需要向前看, 这就是“0”的含义

LR(1)语法分析

SLR(1)分析表

Simple

SLR(1) 分析表

状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2	r2	s7		r2	r2				
3	r4	r4		r4	r4				
4	s5			s4			8	2	3
5	r6	r6		r6	r6				
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9	r1	s7		r1	r1				
10	r3	r3		r3	r3				
11	r5	r5		r5	r5				

归约:

(3) $[k : A \rightarrow \alpha] \in I_i \wedge A \neq S' \implies \forall t \in \text{FOLLOW}(A). \text{ACTION}[i, t] = rk$

如果文法 G 的 $SLR(1)$ 分析表是**无冲突**的, 则 G 是 $SLR(1)$ 文法

例题

题目 1 ([10 = 1 + 2 + 2 + 3 + 2 分])

给定下述文法 G ,

$$L \rightarrow LP \quad (1)$$

$$L \rightarrow P \quad (2)$$

$$P \rightarrow (P) \quad (3)$$

$$P \rightarrow () \quad (4)$$

(1) 简述 G 所对应的语言;

(2) 为 G 构造 $LR(0)$ 自动机;

注意: 为了尽量统一状态编号, 便于批改, 当计算 CLOSURE 时, 请按照文法编号大小顺序加入新项。当计算 $\text{GOTO}(I, X)$ 时, 请按照 I 中项的出现顺序依次考虑可能的转移符号 X 。

要求: 给出初始状态 I_0 的计算方法以及 $\text{GOTO}(I_0, ())$ 的计算方法。

(3) 为该文法设计 $LR(0)$ 分析表; 该文法是 $LR(0)$ 文法吗? 请说明理由。

(4) 为该文法设计 $SLR(1)$ 分析表; 该文法是 $SLR(1)$ 文法吗? 请说明理由。

要求: 请说明归约的设置条件。

(5) 如果该文法是 $SLR(1)$ 文法, 请给出识别输入串 $((()))$ 时自动机所经历的状态 (编号)。

解答:

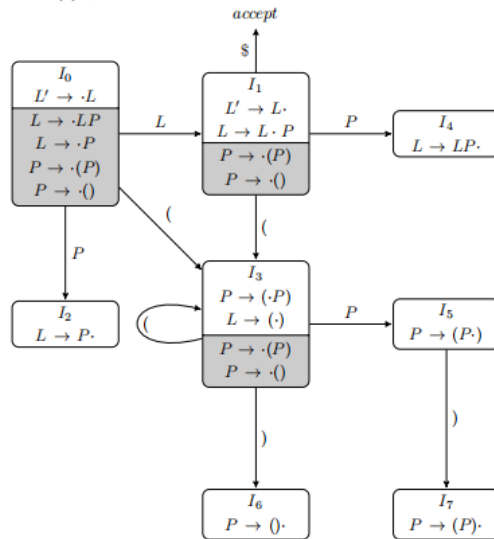
(1) 所有类似 $((\dots))$ 的小括号串的串行组合, 不包括 $((()))$ 这种类型

(2) 增加产生式 $L' \rightarrow L$

1. 计算 I_0 :

初始: $I_0 = \text{CLOSURE}(\{[L' \rightarrow \cdot L]\})$

- $[L' \rightarrow \cdot L]$ 应用 (1)(2): $I_0 = \{[L' \rightarrow \cdot L], [L \rightarrow \cdot LP], [L \rightarrow \cdot P]\}$
 $[L' \rightarrow \cdot P]$ 应用 (3)(4):
 $I_0 = \{[L' \rightarrow \cdot L], [L \rightarrow \cdot LP], [L \rightarrow \cdot P], [P \rightarrow \cdot (P)], [P \rightarrow \cdot ()]\}$
 无法再添项, 完毕。
2. 计算 $GOTO(I_0, ($)
- I_0 中含 $\cdot ($ 的项集: $\{[P \rightarrow \cdot (P)], [P \rightarrow \cdot ()]\}$
 - 计算 $CLOSURE(\{[P \rightarrow \cdot (P)], [P \rightarrow \cdot ()]\})$, 与 I_0 计算方法相同
 - $GOTO(I_0, (= \{[P \rightarrow \cdot (P)], [P \rightarrow \cdot ()], [P \rightarrow \cdot (P)], [P \rightarrow \cdot ()]\}$
3. $LR(0)$ 状态机



(3)

	ACTION			GOTO	
	(	)	\$	L	P
0	s3			g1	g2
1	s3		accept		g4
2	r2	r2	r2		
3	s3	s6			g5
4	r1	r1	r1		
5		s7			
6	r4	r4	r4		
7	r3	r3	r3		

该文法是 $LR(0)$ 文法, 因为 $LR(0)$ 分析表没有冲突

	ACTION			GOTO	
	(	)	\$	L	P
0	s3			g1	g2
1	s3		accept		g4
2	r2		r2		
3	s3	s6			g5
4	r1		r1		
5		s7			
6	r4	r4	r4		
7	r3	r3	r3		

$FOLLOW(L) = \{ (, \$ \}$, $FOLLOW(P) = \{ (,), \$ \}$

说明: 按 $X \rightarrow \gamma$ 规约时, 当前分析的非终结符号 a 要满足: $a \in FOLLOW(X)$

该文法是 $SLR(1)$ 文法, 因为 $SLR(1)$ 分析表没有冲突

(4)

初始状态: 0

PATH: $0 \rightarrow 3 \rightarrow 3 \rightarrow 6 \rightarrow 5 \rightarrow 7 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 6 \rightarrow 4 \rightarrow 1 \rightarrow \text{accept}$

用栈表示: $0 \xrightarrow{\text{移入} (} 03 \xrightarrow{\text{移入} (} 033 \xrightarrow{\text{移入} (} 0336 \xrightarrow{\text{按 4 规约}} 035 \xrightarrow{\text{移入} (} 0357 \xrightarrow{\text{按 3 规约}} 02 \xrightarrow{\text{按 2 规约}}$

$01 \xrightarrow{\text{移入} (} 013 \xrightarrow{\text{移入} (} 0136 \xrightarrow{\text{按 4 规约}} 014 \xrightarrow{\text{按 1 规约}} 01 \rightarrow \text{accept}$

题目4 ANTLR4与“优先级上升算法”

优先级上升算法

例子1	例子2
对应于一段递归函数 $\text{expr}(\text{int } _p)$ <pre> stat: expr > E[0] = expr[int _p] INT ID while { ({4 >= \$_p}? '*' expr[5] {3 >= \$_p}? '+' expr[4]) * } ; </pre> match ID INT while { match '*'; E(5) match '+'; E(4) $(1+2)+3$ $1+2+3$ $1+2*3$ $1*2+3$	对应于一段递归函数 $\text{expr}(\text{int } _p)$ <pre> expr[int _p] : (INT ID) ({4 >= \$_p}? '*' expr[5] {3 >= \$_p}? '+' expr[4])* ; </pre> $1+2+3$ $1+2*3$ $1*2+3$

- 函数的参数 $_p$ 代表优先级，加法的优先级为3，乘法的优先级为4
- 初始调用 $_p$ 的值为0

常见的文法改造

带括号的文法改造

parser-allstar/LRExprParen.g4

```

stat: expr ';' EOF;

expr: expr '*' expr
    | expr '+' expr
    | '(' expr ')'
    | INT
    | ID
    ;

expr[int _p]
: ( '(' expr[0] ')'
  | INT
  | ID
  )
( {5 >= $_p}? '*' expr[6]
| {4 >= $_p}? '+' expr[5]
)*
;

```

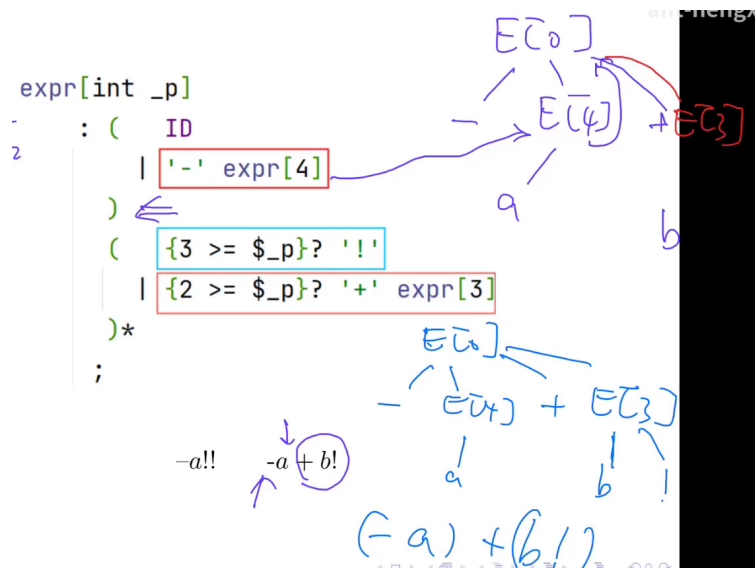
- 不是左递归文法，和终结符一样对待
- 和从外部调用(例如从 `stat` 调用一样)，优先级设为0

阶乘符号、负号的文法改造

- 左结合：优先级上升
- 右结合：优先级不变

改造前	改造后
parser-allstar/LRExprUS.g4 <pre> stat: expr ';' EOF; expr: '-' expr expr '!' expr '+' expr ID ; </pre> 4 - 3 ! 2 + 1	<pre> expr[int _p] : (ID '-' expr[4]) ({3 >= \$_p}? '!' {2 >= \$_p}? '+' expr[3])* ; </pre> 4 - 3 ! 2 + 1

画出给定输入 $(-a + b!)$ 在改造后的文法下对应的语法分析树



幂运算（二元右结合）

```

stat : expr ';' EOF ;
expr : <assoc = right> expr '^' expr
      | expr '+' expr
      | INT
      ;

expr[int _p]
: ( INT
  | {3 >= $_p}? '^' expr[3]
  | {2 >= $_p}? '+' expr[3]
  ) *
;

```

例题

考虑 ANTLR4 改造后的如下 `expr` 文法（上层规则为 `stat : expr[0] ';' ;`）:

```

expr[int _p]
: ( INT
  | ID
  )
  ( {4 >= $_p}? '*' expr[5]
    | {3 >= $_p}? '+' expr[4]
  ) *
;

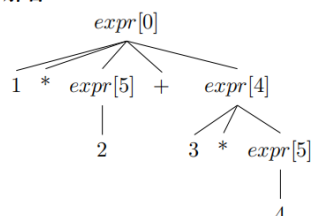
```

请给出 $1 * 2 + 3 + 4 * 5$ 在该文法下对应的语法分析树^③。请给出关键的解释，点到即可，不必面面俱到^④。

③ 该算法称为“优先级上升（Climbing）”算法。

④ 今天课上讲解时遗漏了一个关键点，课后有同学问到了。下次课，我们再回顾一下。

解答：



初始为 `expr[0]`，匹配 1。{4>=0}? 为真，匹配 *。递归调用 `expr[5]`，匹配 2。{3>=5}? 为假，`expr[5]` 返回。{3>=0}? 为真，`expr[0]` 匹配 +。后面的过程同理。

题目5（语法制导的翻译）

属性文法（*Attribute Grammar*）：为上下文无关文法赋予语义

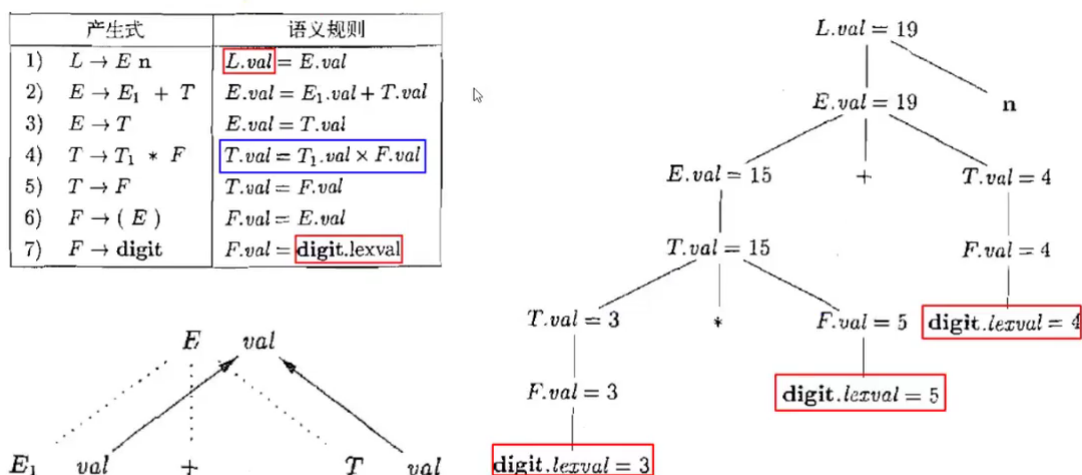
综合属性

父节点的 val 属性依赖于子节点的 val 属性，这种属性称为**综合属性**

Definition (综合属性 (Synthesized Attribute))

节点 N 上的**综合属性**只能通过 N 的子节点或 N 本身的属性来定义。

依赖图用于确定一棵给定的语法分析树中各个属性实例之间的依赖关系



S 属性定义的依赖图刻画了属性实例之间**自底向上**的信息流动

在LL语法分析器中，递归下降函数 A 返回时，计算相应节点 A 的综合属性值。

交互式迷你计算器（ANTLR4中的语法规则**不要求**，仅作为举例使用）：

```
Expr.g4  ExprAG.g4  expfrag.txt
7
8  @parser::members {
9      Map<String, Integer> memory = new HashMap<>();
10
11      int eval(int left, int right, int op) {
12          switch (op) {
13              case ADD : return left + right;
14              case SUB : return left - right;
15              case MUL : return left * right;
16              case DIV : return left / right;
17              default : return 0;
18          }
19      }
20
21
22      prog : stat+ ;
23
```

```

23
24 stat : expr      { System.out.println($expr.val); }
25      | ID '=' expr { memory.put($ID.text, $expr.val); }
26      ;
27
28 expr returns [int val]
29 : l = expr op = ('*' | '/') r = expr { $val = eval($l.val, $r.val, $op.type); }
30   | l = expr op = ('+' | '-') r = expr { $val = eval($l.val, $r.val, $op.type); }
31   | '(' expr ')'                    { $val = $expr.val; }
32   | ID                              { $val = memory.getOrDefault($ID.text, 0); }
33   | INT                             { $val = $INT.int; }
34   ;
35

```

补充: Definition (S 属性定义 (S -Attributed Definition))
 如果一个 SDD 的每个属性都是综合属性, 则它是 S 属性定义。

继承属性

Definition (继承属性 (Inherited Attribute))

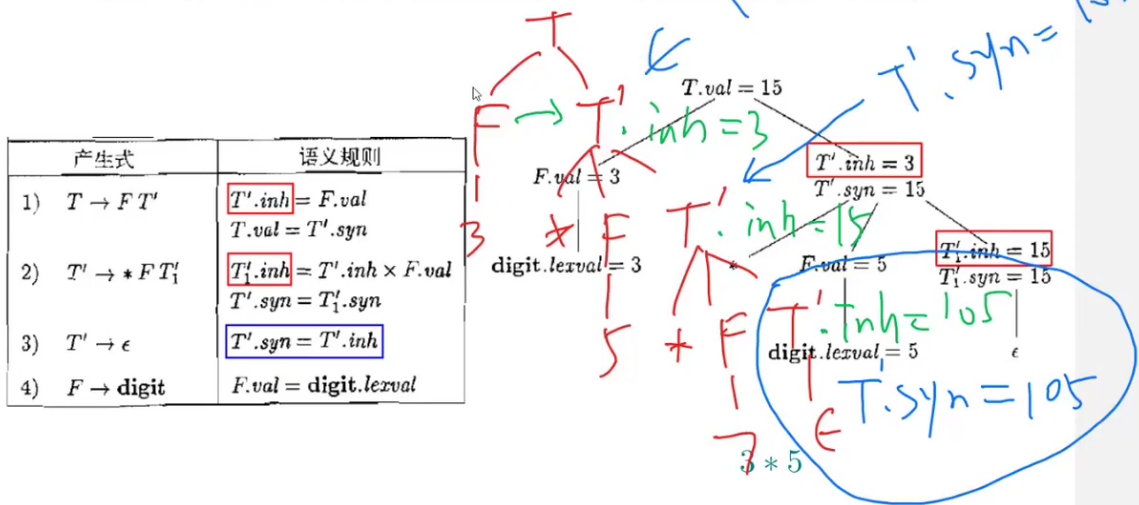
节点 N 上的**继承属性**只能通过 N 的父节点、 N 本身和 N 的兄弟节点上的属性来定义。

示例1

类型声明文法	语法分析树												
类型声明文法举例 <table border="1"> <thead> <tr> <th>产生式</th><th>语义规则</th></tr> </thead> <tbody> <tr> <td>1) $D \rightarrow T L$</td><td>$L.inh = T.type$</td></tr> <tr> <td>2) $T \rightarrow \text{int}$</td><td>$T.type = \text{integer}$</td></tr> <tr> <td>3) $T \rightarrow \text{float}$</td><td>$T.type = \text{float}$</td></tr> <tr> <td>4) $L \rightarrow L_1, \text{id}$</td><td>$L_1.inh = L.inh$ $\text{addType}(\text{id.entry}, L.inh)$</td></tr> <tr> <td>5) $L \rightarrow \text{id}$</td><td>$\text{addType}(\text{id.entry}, L.inh)$</td></tr> </tbody> </table> $\text{float id}_1, \text{id}_2, \text{id}_3$	产生式	语义规则	1) $D \rightarrow T L$	$L.inh = T.type$	2) $T \rightarrow \text{int}$	$T.type = \text{integer}$	3) $T \rightarrow \text{float}$	$T.type = \text{float}$	4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $\text{addType}(\text{id.entry}, L.inh)$	5) $L \rightarrow \text{id}$	$\text{addType}(\text{id.entry}, L.inh)$	$L.inh$ 将声明的类型沿着标识符列表向下传递 $\text{float id}_1, \text{id}_2, \text{id}_3$
产生式	语义规则												
1) $D \rightarrow T L$	$L.inh = T.type$												
2) $T \rightarrow \text{int}$	$T.type = \text{integer}$												
3) $T \rightarrow \text{float}$	$T.type = \text{float}$												
4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $\text{addType}(\text{id.entry}, L.inh)$												
5) $L \rightarrow \text{id}$	$\text{addType}(\text{id.entry}, L.inh)$												

示例2

继承属性 $T'.inh$ 用于在表达式中从左向右传递中间计算结果



Definition (L 属性定义 (L -Attributed Definition))

如果一个 SDD 的每个属性

- (1) 要么是综合属性,
- (2) 要么是继承属性, 但是它的规则满足如下限制:

对于产生式 $A \rightarrow X_1 X_2 \dots X_n$ 及其对应规则定义的继承属性 $X_i.a$, 则这个规则只能使用

 - (a) 和产生式头 A 关联的继承属性;
 - (b) 位于 X_i 左边的文法符号实例 $X_1, X_2, \dots, X_{i-1}$ 相关的继承属性或综合属性;
 - (c) 和这个 X_i 的实例本身相关的继承属性或综合属性, 但是在由这个 X_i 的全部属性组成的依赖图中不存在环。

则它是 L 属性定义。

语法制导定义(Syntax-Directed Definition; SDD)

SDD是一个上下文无关文法和属性及规则的结合

为什么叫语法制导: 动作嵌入到语法规则中, 在语法分析阶段完成

数组类型文法举例

文法举例	依赖图						
数组类型文法举例 <table><tr><th>产生式</th></tr><tr><td>$T \rightarrow B C$</td></tr><tr><td>$B \rightarrow \text{int}$</td></tr><tr><td>$B \rightarrow \text{float}$</td></tr><tr><td>$C \rightarrow [\text{num}] C_1$</td></tr><tr><td>$C \rightarrow \epsilon$</td></tr></table> $\text{int}[2][3]$ 类型表达式 $(2, (3, \text{int}))$	产生式	$T \rightarrow B C$	$B \rightarrow \text{int}$	$B \rightarrow \text{float}$	$C \rightarrow [\text{num}] C_1$	$C \rightarrow \epsilon$	继承属性 $C.b$ 将一个基本类型沿着树向下传播 <pre>graph TD Root["T.t = array(2, array(3, integer))"] Bt["B.t = integer"] Cb1["C.b = integer"] Ct1["C.t = array(2, array(3, integer))"] Cb2["C.b = integer"] Ct2["C.t = array(3, integer)"] Cb3["C.b = integer"] Ct3["C.t = integer"] Epsilon["ε"] Root --> Bt Root --> Cb1 Bt --> Int["int"] Cb1 --> Ct1 Ct1 --> Two["[2]"] Ct1 --> Cb2 Cb2 --> Ct2 Ct2 --> Three["[3]"] Ct2 --> Cb3 Cb3 --> Ct3 Ct3 --> Epsilon</pre> $\text{int}[2][3]$ 综合属性 $C.t$ 收集最终得到的类型表达式
产生式							
$T \rightarrow B C$							
$B \rightarrow \text{int}$							
$B \rightarrow \text{float}$							
$C \rightarrow [\text{num}] C_1$							
$C \rightarrow \epsilon$							

有符号二进制数文法

文法与分析	属性定义																
有符号二进制数文法	有符号二进制数 L 属性定义 <table border="1"> <thead> <tr> <th>产生式</th><th>属性规则</th></tr> </thead> <tbody> <tr> <td>1 $Number \rightarrow Sign List$</td><td>$List.position \leftarrow 0$ if $Sign.negative$ then $Number.value \leftarrow -List.value$ else $Number.value \leftarrow List.value$</td></tr> <tr> <td>2 $Sign \rightarrow +$</td><td>$Sign.negative \leftarrow false$</td></tr> <tr> <td>3 $Sign \rightarrow -$</td><td>$Sign.negative \leftarrow true$</td></tr> <tr> <td>4 $List \rightarrow Bit$</td><td>$Bit.position \leftarrow List.position$ $List.value \leftarrow Bit.value$</td></tr> <tr> <td>5 $List_0 \rightarrow List_1 Bit$</td><td>$List_1.position \leftarrow List_0.position + 1$ $Bit.position \leftarrow List_0.position$ $List_0.value \leftarrow List_1.value + Bit.value$</td></tr> <tr> <td>6 $Bit \rightarrow 0$</td><td>$Bit.value \leftarrow 0$</td></tr> <tr> <td>7 $Bit \rightarrow 1$</td><td>$Bit.value \leftarrow 2^{Bit.position}$</td></tr> </tbody> </table>	产生式	属性规则	1 $Number \rightarrow Sign List$	$List.position \leftarrow 0$ if $Sign.negative$ then $Number.value \leftarrow -List.value$ else $Number.value \leftarrow List.value$	2 $Sign \rightarrow +$	$Sign.negative \leftarrow false$	3 $Sign \rightarrow -$	$Sign.negative \leftarrow true$	4 $List \rightarrow Bit$	$Bit.position \leftarrow List.position$ $List.value \leftarrow Bit.value$	5 $List_0 \rightarrow List_1 Bit$	$List_1.position \leftarrow List_0.position + 1$ $Bit.position \leftarrow List_0.position$ $List_0.value \leftarrow List_1.value + Bit.value$	6 $Bit \rightarrow 0$	$Bit.value \leftarrow 0$	7 $Bit \rightarrow 1$	$Bit.value \leftarrow 2^{Bit.position}$
产生式	属性规则																
1 $Number \rightarrow Sign List$	$List.position \leftarrow 0$ if $Sign.negative$ then $Number.value \leftarrow -List.value$ else $Number.value \leftarrow List.value$																
2 $Sign \rightarrow +$	$Sign.negative \leftarrow false$																
3 $Sign \rightarrow -$	$Sign.negative \leftarrow true$																
4 $List \rightarrow Bit$	$Bit.position \leftarrow List.position$ $List.value \leftarrow Bit.value$																
5 $List_0 \rightarrow List_1 Bit$	$List_1.position \leftarrow List_0.position + 1$ $Bit.position \leftarrow List_0.position$ $List_0.value \leftarrow List_1.value + Bit.value$																
6 $Bit \rightarrow 0$	$Bit.value \leftarrow 0$																
7 $Bit \rightarrow 1$	$Bit.value \leftarrow 2^{Bit.position}$																

例题

题目 1

考虑如下文法 G ,

$P \rightarrow D$
 $D \rightarrow D ; D$
 $D \rightarrow T \text{ id}$
 $D \rightarrow \text{func id } \{D\}$
 $T \rightarrow \text{int}$

请设计语法制导的翻译方案, 完成下列任务。你需要自行定义合适的属性。

- 打印程序 P 一共声明了多少个 id (包括 $T \text{ id}$ 与 func id)。
例如, 句子 $\text{int id} ; \text{func id } \{\text{int id}\}$ 中共声明了 3 个 id 。
- 打印程序 P 中每个变量 id (不考虑函数名 id) (在 $\{\}$ 中) 的嵌套深度。
例如, 在句子 $\text{int id} ; \text{func id } \{\text{int id}\}$ 中, 第一个变量 id 的嵌套深度为 0, 第二个变量 id 的嵌套深度为 1。

参考答案

解答:

(1) *count* 表示 id 的个数

$P \rightarrow D \{print(D.count)\}$
$D \rightarrow D_1 ; D_2 \{D.count = D_1.count + D_2.count\}$
$D \rightarrow T \ id \ {D.count = 1}$
$D \rightarrow func \ id \ {D_1} \ {D.count = D_1.count + 1}$
$T \rightarrow int \ \{\}$

(2) *depth* 表示 id 的深度

$P \rightarrow \{D.depth = 0\} D$
$D \rightarrow \{D_1.depth = D.depth; D_2.depth = D.depth\} D_1 ; D_2$
$D \rightarrow T \ id \ \{print(D.depth)\}$
$D \rightarrow \{D_1.depth = D.depth + 1\} func \ id \ \{D_1\}$
$T \rightarrow int \ \{\}$

T2

题目 1 ([10 = 5 + 5 分])

以下文法 G 描述了带可选小数部分的二进制数。我们希望通过属性文法计算二进制数对应的十进制表示。

$$N \rightarrow L.L$$

$$N \rightarrow L$$

$$L \rightarrow LB$$

$$L \rightarrow B$$

$$B \rightarrow 1$$

$$B \rightarrow 0$$

(1) 请给出一种 S 属性翻译方案。

提示: $123 = (1 \times 10 + 2) \times 10 + 3$

(2) 请给出一种 L 属性翻译方案。

参考答案

(1) w 表示二进制串宽度

$N \rightarrow L_1.L_2\{N.val = L_1.val + 2^{-L_2.w} * L_2.val\}$
$N \rightarrow L\{N.val = L.val\}$
$L \rightarrow L_1B\{L.w = L_1.w + 1, L.val = L_1.val * 2 + B.val\}$
$L \rightarrow B\{L.w = 1, L.val = B.val\}$
$B \rightarrow 1\{B.val = 1\}$
$B \rightarrow 0\{B.val = 0\}$

(2) $L.val$ 的值由 $B.val$ 获得, $L.int = true$ 表示 L 是整数部分, $L.w$ 表示 L 的二进制数宽度, $B.inhVal$ 表示 B 左边的串的二进制整数值, $B.inhW$ 表示 B 左边的二进制位数

$N \rightarrow \{L_1.int = true\}L_1.\{L_2.int = false\}L_2\{N.val = L_1.val + L_2.val\}$
$N \rightarrow \{L.int = true\}L\{N.val = L.val\}$
$L \rightarrow \{L_1.int = L.int\}L_1\{B.inhVal = L_1.val, B.inhW = L_1.w, B.inhInt = L.int\}$ $B\{L.w = L_1.w + 1, L.val = B.val\}$
$L \rightarrow \{B.inhVal = 0, B.inhW = 0, B.inhInt = L.int\}B\{L.w = 1, L.val = B.val\}$
$B \rightarrow 1 \{ \text{if } B.inhInt == true$ $\quad \text{then } B.val = B.inhVal * 2 + 1$ $\quad \text{else } B.val = B.inhVal + 2^{-B.inhW-1} \}$
$B \rightarrow 0 \{ \text{if } B.inhInt == true$ $\quad \text{then } B.val = B.inhVal * 2$ $\quad \text{else } B.val = B.inhVal \}$

题目6（目标代码生成）

RISC-V 调用约定 (Calling Convention)

Registers	Symbolic names	Description	Saver
x0	zero	Hardwired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporary registers	Caller
x8-x9	s0-s1	Saved registers	Callee
x10-x11	a0-a1	Function arguments and return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporary registers	Caller

阶乘程序示例（过程调用与递归）

```

1  .text
2  .global main
3
4  factorial:
5      beqz a0, base_case
6
7      addi sp, sp, -8
8      sw a0, 4(sp)
9      sw ra, 0(sp)
10
11     # n > 0: n * factorial(n - 1)
12     addi a0, a0, -1
13     call factorial
14     mv t0, a0
15
16     lw a0, 0(sp)
17     lw ra, 0(sp)
18     addi sp, sp, 8
19
20     mul a0, a0, t0
21     j end
22
23 base_case:
24     li a0, 1
    
```



```

25
26 end:
27     ret
28
29 ##### main #####
30 .data
31 n: .word 10
32
33 .text
34 main:
35     lw a0, n
36     call factorial

```

MIT 6.191 (6.004) ISA Reference Card: Instructions

Instruction	Syntax	Description	Execution
LUI	lui rd, luiConstant	Load Upper Immediate	reg[rd] <= luiConstant « 12
JAL	jal rd, label	Jump and Link	reg[rd] <= pc + 4 pc <= label
JALR	jalr rd, offset(rs1)	Jump and Link Register	reg[rd] <= pc + 4 pc <= {(reg[rs1] + offset)[31:1], 1'b0}
BEQ	beq rs1, rs2, label	Branch if =	pc <= (reg[rs1] == reg[rs2]) ? label: pc + 4
BNE	bne rs1, rs2, label	Branch if ≠	pc <= (reg[rs1] != reg[rs2]) ? label: pc + 4
BLT	blt rs1, rs2, label	Branch if < (Signed)	pc <= (reg[rs1] < _s reg[rs2]) ? label: pc + 4
BGE	bge rs1, rs2, label	Branch if ≥ (Signed)	pc <= (reg[rs1] >= _s reg[rs2]) ? label: pc + 4
BLTU	bltu rs1, rs2, label	Branch if < (Unsigned)	pc <= (reg[rs1] < _u reg[rs2]) ? label: pc + 4
BGEU	bgeu rs1, rs2, label	Branch if ≥ (Unsigned)	pc <= (reg[rs1] >= _u reg[rs2]) ? label: pc + 4
LW	lw rd, offset(rs1)	Load Word	reg[rd] <= mem[reg[rs1] + offset]
SW	sw rs2, offset(rs1)	Store Word	mem[reg[rs1] + offset] <= reg[rs2]
ADDI	addi rd, rs1, constant	Add Immediate	reg[rd] <= reg[rs1] + constant
SLTI	slti rd, rs1, constant	Compare < Immediate (Signed)	reg[rd] <= (reg[rs1] < _s constant) ? 1 : 0
SLTIU	sltiu rd, rs1, constant	Compare < Immediate (Unsigned)	reg[rd] <= (reg[rs1] < _u constant) ? 1 : 0
XORI	xori rd, rs1, constant	Xor Immediate	reg[rd] <= reg[rs1] ^ constant
ORI	ori rd, rs1, constant	Or Immediate	reg[rd] <= reg[rs1] constant
ANDI	andi rd, rs1, constant	And Immediate	reg[rd] <= reg[rs1] & constant
SLLI	slli rd, rs1, shamt	Shift Left Logical Immediate	reg[rd] <= reg[rs1] « shamt
SRLI	srl rd, rs1, shamt	Shift Right Logical Immediate	reg[rd] <= reg[rs1] » _u shamt
SRAI	srai rd, rs1, shamt	Shift Right Arithmetic Immediate	reg[rd] <= reg[rs1] » _s shamt
ADD	add rd, rs1, rs2	Add	reg[rd] <= reg[rs1] + reg[rs2]
SUB	sub rd, rs1, rs2	Subtract	reg[rd] <= reg[rs1] - reg[rs2]
SLL	sll rd, rs1, rs2	Shift Left Logical	reg[rd] <= reg[rs1] « reg[rs2][4:0]
SLT	slt rd, rs1, rs2	Compare < (Signed)	reg[rd] <= (reg[rs1] < _s reg[rs2]) ? 1 : 0
SLTU	sltu rd, rs1, rs2	Compare < (Unsigned)	reg[rd] <= (reg[rs1] < _u reg[rs2]) ? 1 : 0
XOR	xor rd, rs1, rs2	Xor	reg[rd] <= reg[rs1] ^ reg[rs2]
SRL	srl rd, rs1, rs2	Shift Right Logical	reg[rd] <= reg[rs1] » _u reg[rs2][4:0]
SRA	sra rd, rs1, rs2	Shift Right Arithmetic	reg[rd] <= reg[rs1] » _s reg[rs2][4:0]
OR	or rd, rs1, rs2	Or	reg[rd] <= reg[rs1] reg[rs2]
AND	and rd, rs1, rs2	And	reg[rd] <= reg[rs1] & reg[rs2]

Note: *luiConstant* is a 20-bit value. *offset* and *constant* are signed 12-bit values that are sign-extended to 32-bit values. *label* is a 32-bit memory address or its alias name. *shamt* is a 5-bit unsigned shift amount.

MIT 6.191 (6.004) ISA Reference Card: Pseudoinstructions

Pseudoinstruction	Description	Execution
li rd, liConstant	Load Immediate	reg[rd] <= liConstant
mv rd, rs1	Move	reg[rd] <= reg[rs1] + 0
not rd, rs1	Logical Not	reg[rd] <= reg[rs1] ^ -1
neg rd, rs1	Arithmetic Negation	reg[rd] <= 0 - reg[rs1]
j label	Jump	pc <= label
jal label	Jump and Link (with <i>ra</i>)	reg[ra] <= pc + 4 pc <= label
call label		
jr rs	Jump Register	pc <= reg[rs1] & ~1
jalr rs	Jump and Link Register (with <i>ra</i>)	reg[ra] <= pc + 4 pc <= reg[rs1] & ~1
ret	Return from Subroutine	pc <= reg[ra]
bgt rs1, rs2, label	Branch > (Signed)	pc <= (reg[rs1] > _s reg[rs2]) ? label : pc + 4
bte rs1, rs2, label	Branch ≤ (Signed)	pc <= (reg[rs1] <= _s reg[rs2]) ? label : pc + 4
bgtu rs1, rs2, label	Branch > (Unsigned)	pc <= (reg[rs1] > _s reg[rs2]) ? label : pc + 4
bleu rs1, rs2, label	Branch ≤ (Unsigned)	pc <= (reg[rs1] <= _s reg[rs2]) ? label : pc + 4
beqz rs1, label	Branch = 0	pc <= (reg[rs1] == 0) ? label : pc + 4
bnez rs1, label	Branch ≠ 0	pc <= (reg[rs1] != 0) ? label : pc + 4
bltz rs1, label	Branch < 0 (Signed)	pc <= (reg[rs1] < _s 0) ? label : pc + 4
bgez rs1, label	Branch ≥ 0 (Signed)	pc <= (reg[rs1] >= _s 0) ? label : pc + 4
bgtz rs1, label	Branch > 0 (Signed)	pc <= (reg[rs1] > _s 0) ? label : pc + 4
blez rs1, label	Branch ≤ 0 (Signed)	pc <= (reg[rs1] <= _s 0) ? label : pc + 4

Note: *liConstant* is a 32-bit value.